Academia\BackEnd\Academia Server\Academia_Server.c

```c
/*
Harshul Gupta, Lawrence Mao, Suengho Kim, Dmitrii Buianov
CS50 C Project
Academia - Local Web Server
*/
#define WIN32_LEAN_AND_MEAN
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <winsock2.h>
#include <ws2tcpip.h>
#include <sqlite3.h>

#include "Database.h"
#include "Academia Auth/Academia_Auth.h"
#include "Academia Student/Academia_Student.h"
#include "Academia Staff/Academia_Staff.h"
#include "Academia Parent/Academia_Parents.h"

#pragma comment(lib, "ws2_32.lib")
#pragma comment(lib, "winsqlite3.lib")

#define PORT 3000
#define BUFFER_SIZE 65536

static sqlite3 *db = NULL;

void send_response(SOCKET client, int status, const char *status_text, const char *content_type, const char *body) {
    char header[2048];
    int length = body ? (int)strlen(body) : 0;
    snprintf(header, sizeof(header),
             "HTTP/1.1 %d %s\r\nContent-Type: %s\r\nContent-Length: %d\r\nAccess-Control-Allow-Origin: *\r\nConnection: close\r\n\r\n",
             status, status_text, content_type, length);
    send(client, header, (int)strlen(header), 0);
    if (length > 0) send(client, body, length, 0);
}

void send_file(SOCKET client, const char *filepath, const char *content_type) {
    FILE *fp = fopen(filepath, "rb");
    if (!fp) {
        send_response(client, 404, "Not Found", "text/html", "<h1>404 - File Not Found</h1>");
        return;
    }
    fseek(fp, 0, SEEK_END);
    long size = ftell(fp);
```

```
    fseek(fp, 0, SEEK_SET);

    char header[1024];
    snprintf(header, sizeof(header), "HTTP/1.1 200 OK\r\nContent-Type: %s\r\nContent-Length: %ld\r\nConnection: close\r\n\r\n", content_type, size);
    send(client, header, (int)strlen(header), 0);

    char chunk[8192];
    size_t read;
    while ((read = fread(chunk, 1, sizeof(chunk), fp)) > 0) {
        send(client, chunk, (int)read, 0);
    }
    fclose(fp);
}

// Copies the text of "key":"value" from the request body into out.
void get_value(const char *body, const char *key, char *out, int size) {
    char pattern[64];
    snprintf(pattern, sizeof(pattern), "\"%s\"", key);
    out[0] = '\0';

    char *start = strstr(body, pattern);
    if (!start) return;
    start = strchr(start + strlen(pattern), '"');
    if (!start) return;
    start++;

    int i = 0;
    while (*start && *start != '"' && i < size - 1) {
        out[i++] = *start++;
    }
    out[i] = '\0';
}

// The portal pages send the login token in the X-Auth-Token header.
void get_token(const char *request, char *token, int size) {
    token[0] = '\0';
    char *start = strstr(request, "X-Auth-Token: ");
    if (!start) return;
    start += 14;

    int i = 0;
    while (*start && *start != '\r' && *start != '\n' && i < size - 1) {
        token[i++] = *start++;
    }
    token[i] = '\0';
}

void handle_client_request(SOCKET client) {
    char buffer[BUFFER_SIZE];
```

```
    int received = recv(client, buffer, sizeof(buffer) - 1, 0);
    if (received <= 0) return;
    buffer[received] = '\0';

    char method[16], path[512];
    sscanf(buffer, "%15s %511s", method, path);

    // Login: checks the username, password and role against the database
    if (strcmp(path, "/api/login") == 0) {
        char *body = strstr(buffer, "\r\n\r\n");
        char username[64], password[64], role[16];
        char response[1024], token[128], name[128];
        int user_id;

        if (!body) return;
        get_value(body, "username", username, sizeof(username));
        get_value(body, "password", password, sizeof(password));
        get_value(body, "role", role, sizeof(role));

        if (auth_login(db, username, password, response, sizeof(response), token, sizeof(token), &user_id, role, sizeof(role), name, sizeof(name))) {
            send_response(client, 200, "OK", "application/json", response);
        } else {
            send_response(client, 401, "Unauthorized", "application/json", response);
        }
        return;
    }

    // Data pages: only a signed in user with the right role gets the data
    if (strcmp(path, "/api/student-data") == 0 || strcmp(path, "/api/staff-data") == 0 || strcmp(path, "/api/parent-data") == 0) {
        char token[128], role[16], name[128];
        int user_id;

        get_token(buffer, token, sizeof(token));
        if (!auth_check(token, &user_id, role, sizeof(role), name, sizeof(name))) {
            send_response(client, 401, "Unauthorized", "application/json", "{\"success\":false}");
            return;
        }

        if (strcmp(path, "/api/student-data") == 0 && strcmp(role, "student") == 0) {
            update_student_frontend_state(user_id);
            send_file(client, "Academia/FrontEnd/JS/student_states.js", "application/javascript");
        } else if (strcmp(path, "/api/staff-data") == 0 && strcmp(role, "staff") == 0) {
            update_staff_frontend_state(user_id);
            send_file(client, "Academia/FrontEnd/JS/staff_states.js", "application/javascript");
        } else if (strcmp(path, "/api/parent-data") == 0 && strcmp(role, "parent") == 0) {
            update_parent_frontend_state(user_id);
```

```
            send_file(client, "Academia/FrontEnd/JS/parents_states.js", "application/javascript");
        } else {
            send_response(client, 403, "Forbidden", "application/json", "{\"success\":false}");
        }
        return;
    }

    // The landing page links to its sister pages without a folder, so it has to be opened from /HTML/
    if (strcmp(path, "/") == 0 || strcmp(path, "/index.html") == 0) {
        const char *redirect = "HTTP/1.1 302 Found\r\nLocation: /HTML/Academia_Landing.html\r\nContent-Length: 0\r\nConnection: close\r\n\r\n";
        send(client, redirect, (int)strlen(redirect), 0);
        return;
    }

    // Never leave the FrontEnd folder
    if (strstr(path, "..")) {
        send_response(client, 403, "Forbidden", "text/html", "<h2>403 Forbidden</h2>");
        return;
    }

    char target[512];
    snprintf(target, sizeof(target), "Academia/FrontEnd%s", path);
    FILE *fp = fopen(target, "rb");
    if (fp) {
        fclose(fp);
        const char *type = "text/html";
        if (strstr(target, ".css")) type = "text/css";
        else if (strstr(target, ".js")) type = "application/javascript";
        else if (strstr(target, ".png")) type = "image/png";
        else if (strstr(target, ".mp4")) type = "video/mp4";
        send_file(client, target, type);
    } else {
        send_response(client, 404, "Not Found", "text/html", "<h2>404 Not Found</h2>");
    }
}

int main(void) {
    printf("=====================================================\n");
    printf("     ACADEMIA SERVER STARTING ON PORT %d\n", PORT);
    printf("=====================================================\n");

    db = open_database();
    if (!db) return 1;

    WSADATA wsa;
    WSAStartup(MAKEWORD(2, 2), &wsa);
    SOCKET server = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
```

```
188
189     struct sockaddr_in addr = {0};
190     addr.sin_family = AF_INET;
191     addr.sin_addr.s_addr = INADDR_ANY;
192     addr.sin_port = htons(PORT);
193
194     bind(server, (struct sockaddr *)&addr, sizeof(addr));
195     listen(server, SOMAXCONN);
196
197     printf("Server running at: http://localhost:%d\n", PORT);
198
199     while (1) {
200         SOCKET client = accept(server, NULL, NULL);
201         if (client == INVALID_SOCKET) continue;
202         handle_client_request(client);
203         closesocket(client);
204     }
205
206     closesocket(server);
207     WSACleanup();
208     close_database(db);
209     return 0;
210 }
```